

Part 4: Understanding Data Sets

Summary

This Part presents an example of reading in a data set, and ensuring the data is fully understood and properly represented prior to any further analysis.

The ability to read in, clean, and understand data sets is a crucial skill. Data do not typically come in a convenient form. Careful consideration should be given to exploring the data to ensure full understanding of the nature of the data set.

Example Data Set

First, we will read in a data set that we will use in our following examples.

Visit the website

https://www.sec.gov/opa/data/market-structure/marketstructuredownloadshtml-by_security.html

and download the data from the first quarter of 2017. Create a directory for working on these examples, and place this file in that directory.

These data come from the U.S. Securities and Exchange Commission repository on market structure. This data set contains daily characteristics of a range of market variables over 5000 equities and ETFs, accumulated over several exchanges.

- 1 Be sure that the working directory is correctly set in R.
- 2 Now use the `read.table()` function to read in the file:

```
> fulldata = read.table("q1_2017_all.csv", sep="," ,  
+                        header=T, quote=" ")
```

- 3 This is a “csv” file, i.e., there are “comma-separated-values”. This leads to
- 4 the use of `sep=","`. The `header=T` argument specifies that the first line
- 5 of the file gives the variable names.

- 1 **Exercise:** What happens if you use `read.table()` in the example above
- 2 without `quote=" "`?

- 3 _____
- 4 _____
- 5 _____
- 6 _____
- 7 _____
- 8 _____
- 9 _____
- 10 _____
- 11 _____
- 12 _____

1 Understanding the Variables

2 Whenever a new data set is encountered, one should fully understand its
3 variables. Specific questions to address for each variable:

- 4 1. Describe the information that is stored in this column.
- 5 2. What type of variable is it? (Timestamp, ordinal factor, categorical fac-
6 tor, ratio scale, other?) Be sure that R represents the variable correctly.
- 7 3. Are there any missing values? What is the source of the missingness?
- 8 4. Are there any extreme/inappropriate values? What is the source of
9 the extreme value?

1 Look at the variables:

```
> names(fulldata)
[1] "Date"           "Security"
[3] "Ticker"         "McapRank"
[5] "TurnRank"       "VolatilityRank"
[7] "PriceRank"      "LitVol..000."
[9] "OrderVol..000." "Hidden"
[11] "TradesForHidden" "HiddenVol..000."
[13] "TradeVolForHidden..000." "Cancels"
[15] "LitTrades"      "OddLots"
[17] "TradesForOddLots" "OddLotVol..000."
[19] "TradeVolForOddLots..000."
```

2 Background on the data can be found in the associated README file, avail-
3 able on Canvas in the “Data Sets” folder in the “Files” page. We will refer-
4 ence portions of it below.

1 Column 1: Date

2 The first column in `fulldata` is a date stamp for the observation. R has
3 special functions for handling dates which will prove useful later when
4 working with time series.

5 The `as.Date()` function transforms into the date format:

```
> print(fulldata$Date[1]) # This is Jan. 3, 2017  
[1] 20170103
```

```
> fulldata$Date = as.Date(as.character(fulldata$Date),  
+                          format="%Y%m%d")
```

6 Note that it is necessary to cast the dates as strings, and then specify the
7 format of those strings.

1 Columns 2 and 3: Security and Ticker

2 The variables `Security` and `Ticker` are already appropriately treated as
3 **factors** by R:

```
> is.factor(fulldata$Sec) & is.factor(fulldata$Tick)  
[1] TRUE
```

4 We also note that `Security` is either `Stock` or `ETF` (Exchange Traded
5 Fund), and that there are over 5,000 equities under consideration:

```
> levels(fulldata$Security)  
[1] "ETF" "Stock"
```

```
> nlevels(fulldata$Ticker)  
[1] 5338
```

1 **Exercise:** How many of the different ticker symbols are ETFs?

2 _____

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

1 **Exercise:** Execute the commands below, and discuss what you find:

```
> length(unique(fulldata$Date))  
> table(table(fulldata$Ticker))  
> which.max(table(fulldata$Ticker))  
> fulldata[duplicated(fulldata[,c(1,3)]),]
```

2 _____

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

- 1 We will remove the duplicated lines using the following command:

```
> fulldata = fulldata[!duplicated(fulldata[,c(1,3)],  
+                               fromLast=TRUE), ]
```

- 2 Note that by using `fromLast=TRUE`, we remove the **first** of each pair of
- 3 duplicates.

1 **Columns 4 through 7: Rank Variables**

- 2 The next four columns provide ranks of the stocks/ETFs with respect to
- 3 key attributes of Market Capitalization, Turnover, Volatility, and Price.
- 4 In general, we would prefer to start with data in its most “raw” format,
- 5 and one could construct variables such as these from that data. Alas, we
- 6 are often left to work with what is available.

- 1 We will change these into **ordered factors** to reflect their ordinal nature:

```
> fulldata$McapRank =  
+   factor(fulldata$McapRank, ordered=TRUE)  
> fulldata$TurnRank =  
+   factor(fulldata$TurnRank, ordered=TRUE)  
> fulldata$VolatilityRank =  
+   factor(fulldata$VolatilityRank, ordered=TRUE)  
> fulldata$PriceRank =  
+   factor(fulldata$PriceRank, ordered=TRUE)
```

- 1 **Columns 8 thru 19: The Count Variables**
- 2 The remaining column in the data set consist of trade and share counts of
- 3 different types.
- 4 Note that some of the volume counts are reported in 1000's of shares, hence
- 5 there are non-integer values among the counts.

- 1 The `summary()` function is a useful first step in understanding the basic
- 2 properties of the variables. A primary concern is the presence of extreme
- 3 or impossible values.

```
> summary(fulldata[,8:19])
```

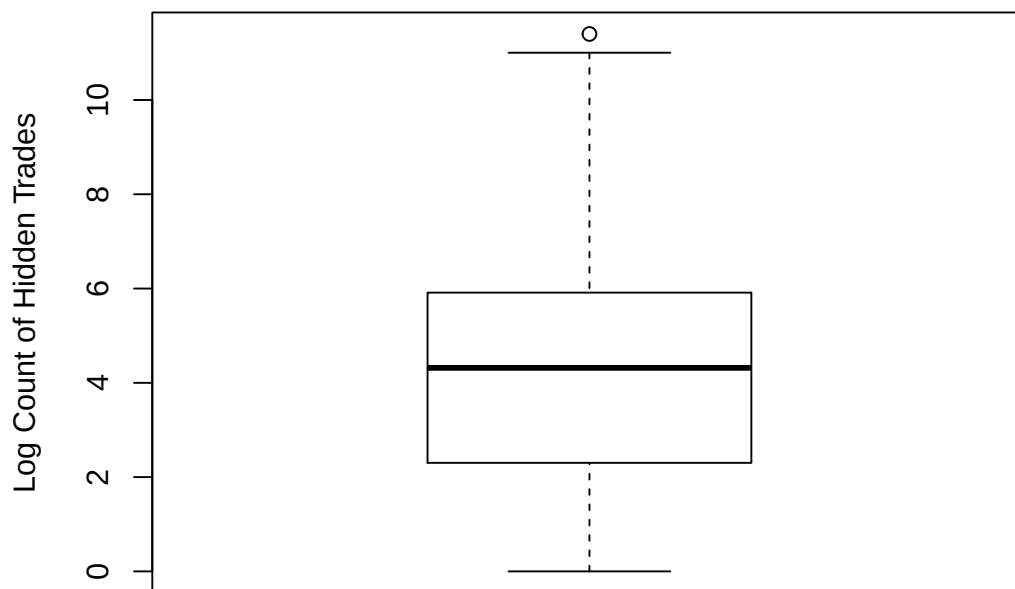
LitVol...000.	OrderVol...000.	Hidden	TradesForHidden
Min. : 0.00	Min. : 0	Min. : -143.0	Min. : 0
1st Qu.: 3.46	1st Qu.: 885	1st Qu.: 10.0	1st Qu.: 41
Median : 44.57	Median : 4003	Median : 75.0	Median : 501
Mean : 431.76	Mean : 35747	Mean : 454.4	Mean : 3476
3rd Qu.: 254.41	3rd Qu.: 15581	3rd Qu.: 370.0	3rd Qu.: 2846
Max. : 119014.06	Max. : 7293901	Max. : 89335.0	Max. : 533874
HiddenVol...000.	TradeVolForHidden...000.	Cancels	
Min. : -7903.64	Min. : 0.00	Min. : 0	
1st Qu.: 1.15	1st Qu.: 5.64	1st Qu.: 3126	
Median : 8.74	Median : 55.31	Median : 15583	
Mean : 66.91	Mean : 498.04	Mean : 73579	
3rd Qu.: 44.02	3rd Qu.: 301.58	3rd Qu.: 60112	
Max. : 14821.94	Max. : 133714.59	Max. : 7607714	

LitTrades	OddLots	TradesForOddLots	OddLotVol...000.
Min. : 0	Min. : 0	Min. : 0	Min. : 0.000
1st Qu.: 23	1st Qu.: 8	1st Qu.: 39	1st Qu.: 0.280
Median : 377	Median : 154	Median : 464	Median : 5.254
Mean : 2708	Mean : 916	Mean : 3115	Mean : 32.492
3rd Qu.: 2186	3rd Qu.: 887	3rd Qu.: 2558	3rd Qu.: 31.083
Max. : 383222	Max. : 130106	Max. : 462414	Max. : 3728.885
TradeVolForOddLots...000.			
Min. : 0.00			
1st Qu.: 5.31			
Median : 49.94			
Mean : 422.78			
3rd Qu.: 260.18			
Max. : 108034.79			

- 1 Of course, graphical tools are useful for understanding the properties of
2 variables. This topic will be covered more fully in the next Part, but here
3 we consider a classic approach: the **boxplot**:

```
> boxplot(log(fulldata$Hidden),  
+         ylab="Log Count of Hidden Trades")
```

- 4 The “box” of the plot extends from the 25th to the 75th percentile of the
5 data, while the solid line in the middle of the box is at the median. The
6 two “arms” extend to the minimum and maximum value, **except** that any
7 value labelled an **outlier** is plotted separately.



2 Missing Data

- 3 Most data sets contain some amount of missing data, for many reasons.

- 1 R uses the special symbol `NA` to represent a missing value.
- 2 As a first step, it is important to recognize how your data set repre-
- 3 sents missing values, so that R handles them properly. The function
- 4 `read.table()` has an argument `na.strings` to allow one to specify
- 5 missing values. **Be careful:** It is not unusual for data sets to use numbers
- 6 such as `-999` to indicate a missing value.
- 7 In a `.csv` file such as that we are using here, it is common to simply have
- 8 blank values to indicate missingness, i.e., there are consecutive commas
- 9 without a value between.

- 1 **Exercise:** Does R appropriately handle cases where a missing value is in-
- 2 dicated using blank values in a `.csv` file?

- 3 _____
- 4 _____
- 5 _____
- 6 _____
- 7 _____
- 8 _____
- 9 _____
- 10 _____
- 11 _____
- 12 _____

- 1 A useful tool is the function `md.pattern()` that is part of package `mice`.
- 2 This function shows the different “patterns” of missingness in the data set,
- 3 and how often they occur.
- 4 Consider the example on the following page, the result of calling

```
> md.pattern(data.frame(fulldata), plot=FALSE)
```

- 5 This shows that there are 325,082 complete cases in the data set. There is
- 6 one case where `VolatilityRank` is missing, and there are 76 cases where
- 7 `McapRank`, `TurnRank` and `PriceRank` are missing.

```

1      Date Security Ticker LitVol..000. OrderVol..000. Hidden
2 325082      1      1      1          1          1      1
3 76      1      1      1          1          1      1
4 1      1      1      1          1          1      1
5      0      0      0          0          0      0
6      TradesForHidden HiddenVol..000. TradeVolForHidden..000. Cancels
7 325082          1          1          1          1
8 76          1          1          1          1
9 1          1          1          1          1
10      0          0          0          0
11      LitTrades OddLots TradesForOddLots OddLotVol..000.
12 325082      1      1          1          1
13 76      1      1          1          1
14 1      1      1          1          1
15      0      0          0          0
16      TradeVolForOddLots..000. VolatilityRank McapRank TurnRank PriceRank
17 325082          1          1          1          1          1
18 76          1          1          0          0          0
19 1          1          0          1          1          1
20      0          1          76          76          76
21
22 325082      0
23 76      3
24 1      1
25      229

```

1 **Exercise:** Inspect the output from

```
> fulldata[!complete.cases(fulldata),]
```

2 What does this say about the cases with missing values?

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10

1 Of course, more sophisticated visualization of the cases with missing data
2 would likely provide more information as to the source of the missingness.
3 In the next Part we will consider such tools.